

GÊMEO DIGITAL DE SEGURANÇA

AMR Fire Detection · Command Center 3D / VR

Relatório Técnico Completo

Projeto	Gêmeo Digital de Segurança Industrial
Evento	Hackathon Ericsson 2026
Data	06 de May de 2026
Stack	Python · Node.js · React 18 · Three.js · WebXR
Modelo IA	YOLOv8/12 (fire / smoke detection)
Repositório	C:/GEMEODIGITAL/

Resumo Executivo

Este documento descreve a arquitetura técnica, as implementações concluídas e as pendências de cada squad do sistema **Gêmeo Digital de Segurança Industrial**. O sistema integra visão computacional por IA (YOLO), backend em tempo real (Node.js + WebSocket) e uma interface 3D/VR imersiva (React Three Fiber) para monitoramento de um robô AMR em uma usina térmica, com localização por otimização matemática (Lagrange + Gradiente) e verificação de consistência com relatórios LLM.

1. Arquitetura do Sistema

O sistema é composto por **três camadas independentes** que comunicam via contratos de dados JSON bem definidos. Cada camada pode ser desenvolvida, testada e substituída de forma isolada:

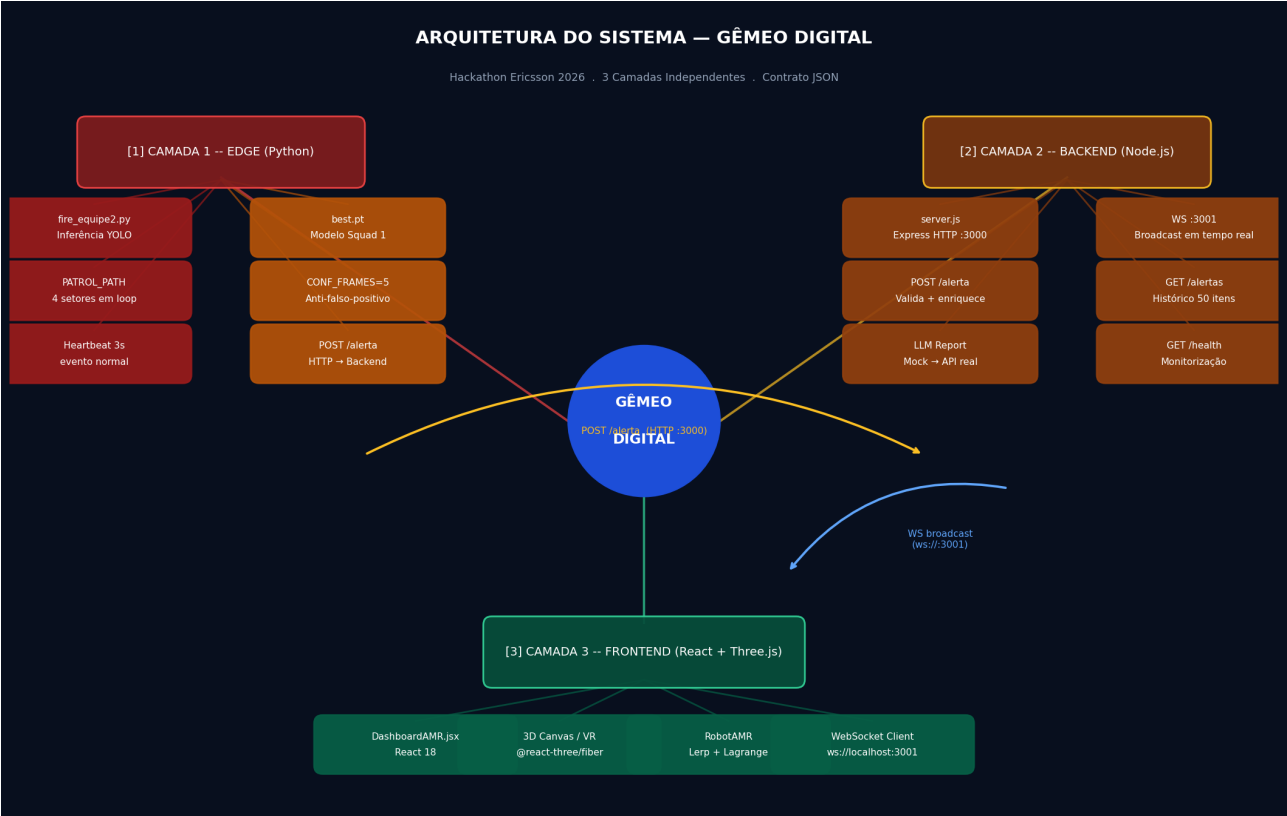


Figura 1 — Arquitetura de 3 camadas e fluxo de dados entre squads

Camada	Tecnologia	Portas	Responsabilidade Principal
■ Edge / IA	Python 3.10 + YOLOv8/12	—	Inferência em tempo real, heartbeat de patrulha a cada 3 s
■ Backend	Node.js + Express + WebSockets	3000 / 3001	Validação do contrato JSON, geração de relatório LLM, broadcast
■ Frontend	React 18 + Three.js + WebXR	5173 (Vite)	Dashboard 3D/VR, robô animado por lerp, verificação Lagrange

1.1 Pipeline de Dados

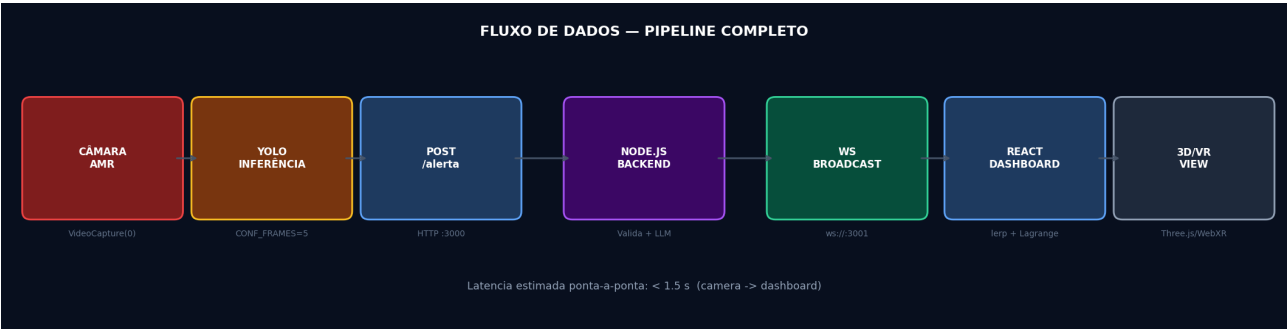


Figura 2 — Pipeline ponta-a-ponta: câmara → YOLO → backend → WS → Three.js

1.2 Contrato JSON — POST /alerta

Todos os dados trafegam neste formato. O campo *localizacao_otimizada.setor* é mapeado para coordenadas 3D pelo frontend via **SECTOR_TO_3D**:

```
{ "id_alerta": "ALRT-1234", "timestamp": "2026-05-06T19:40:00Z",  
  "evento": "fogo | fumaca | normal", "confianca": 0.94,  
  "localizacao_otimizada": { "x": 350, "y": 120,  
    "setor": "Setor D - Caldeiras Químicas" },  
  "llm_prompt": "URGENTE: fogo no Setor D..." }
```

2. Modelo Matemático — Localização por Otimização

A localização precisa do AMR na planta 3D é resolvida por **cinco blocos matemáticos** complementares, todos executados em tempo real no frontend (60 fps via *useFrame*):

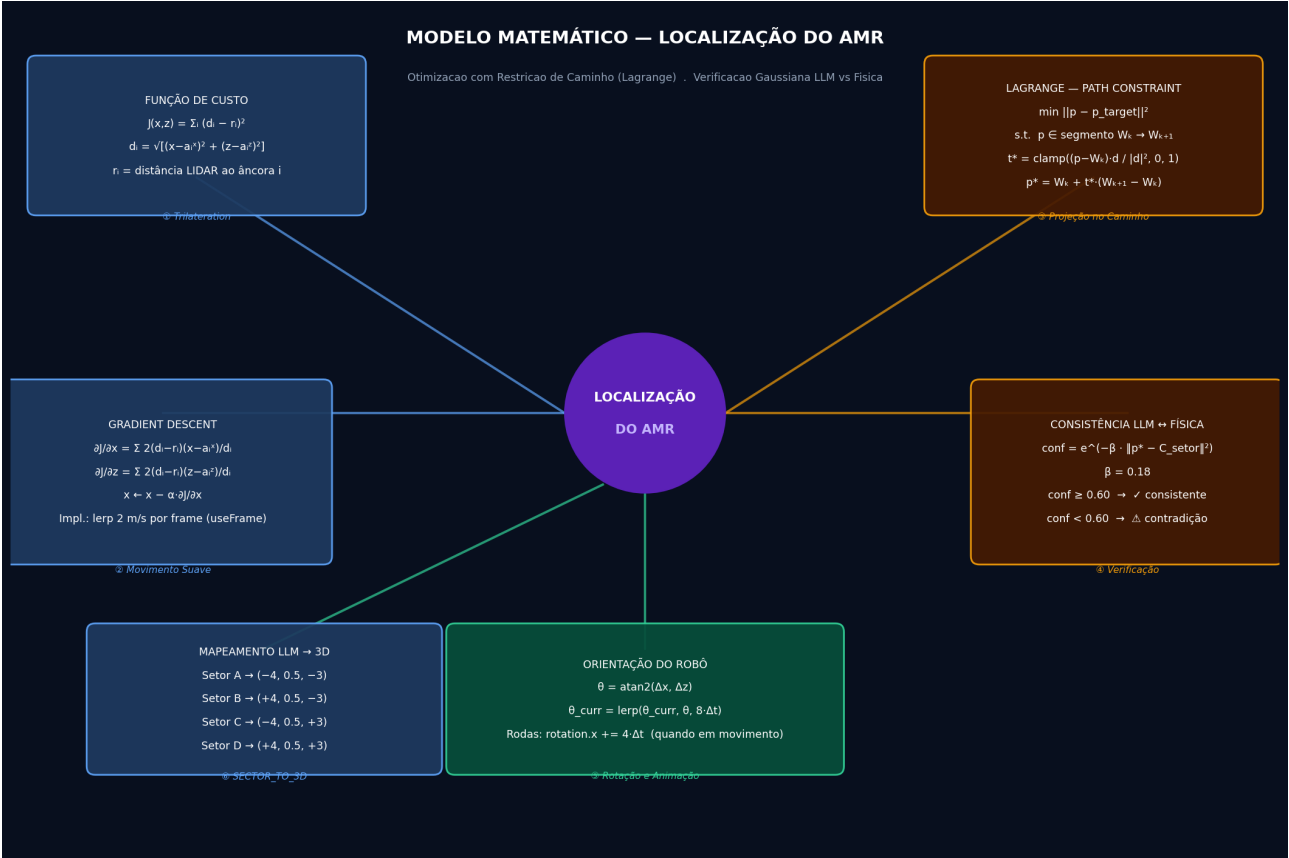


Figura 3 — Mapa mental do modelo matemático de localização do AMR

2.1 Equações Principais

Módulo	Equação Central	Implementação no Código
① Função de Custo (Trilateration)	$J(x,z) = \sum_i (d_i(x,z) - r_i)^2$ $d_i = \sqrt{[(x-a_i)^2 + (z-a_i)^2]}$	Base para estimar posição por múltiplos âncoras LIDAR
② Gradient Descent (movimento lerp)	$p(t+\Delta t) = p(t) + \alpha \cdot (\text{target} - p(t))$ $\alpha = \text{SPEED} \cdot \Delta t / \text{dist}$	useFrame(): lerp adaptativo 2 m/s — sem teleporte
③ Lagrange (path constraint)	$\min \ p - p_{\text{target}}\ ^2$ s.t. $p \in \text{seg. } W_i \rightarrow W_{i+1}$ $t^* = \text{clamp}((p - W_i) \cdot d / d ^2, 0, 1)$	projectOntoPatrolPath() Solução fechada $O(n_{\text{seg}})$
④ Consistência LLM ↔ Física	$\text{conf} = e^{(-\beta \cdot \ p^* - C_{\text{setor}}\ ^2)}$ $\beta = 0.18$ · threshold = 0.60	computeLocationConsistency() ✓/✗ exibido no HUD
⑤ Orientação do robô	$\theta = \text{atan2}(\Delta x, \Delta z)$ $\theta_{\text{curr}} = \text{lerp}(\theta_{\text{curr}}, \theta, 8 \cdot \Delta t)$	group.rotation.y Visor aponta para destino

2.2 Mapeamento Setores → Coordenadas 3D

Setor (texto LLM)	x3d	y3d	z3d	THREE.Vector3
Setor A — Turbinas	-4	0.5	-3	Vector3(-4, 0.5, -3)
Setor B — Geradores	+4	0.5	-3	Vector3(+4, 0.5, -3)
Setor C — Painéis de Controlo	-4	0.5	+3	Vector3(-4, 0.5, +3)
Setor D — Caldeiras Químicas	+4	0.5	+3	Vector3(+4, 0.5, +3)

3. Estado Atual — O que está Implementado

■ Funcionalidade	Camada	Ficheiro	Detalhe Técnico
3D Canvas com GLB da usina	Frontend	DashboardAMR.jsx	usina_interno.glb carregado via useGLTF + Suspense
RobotAMR mesh 3D completo	Frontend	DashboardAMR.jsx	Chassis, cabine, visor, 4 rodas, LIDAR cilíndrico, antena
Movimento suave (lerp)	Frontend	DashboardAMR.jsx	Gradient descent 2 m/s por frame — sem teleporte
Rotação do robô	Frontend	DashboardAMR.jsx	atan2 + lerp angular — visor aponta sempre para destino
Rodas animadas	Frontend	DashboardAMR.jsx	rotation.x += 4·Δt quando distância ao alvo > 0.05
Projeção Lagrangiana no caminho	Frontend	DashboardAMR.jsx	projectOntoPatrolPath() — solução fechada O(n_seg)
Verificação LLM ↔ posição	Frontend	DashboardAMR.jsx	Gaussiana conf = e ^{−0.18·d²} , exibida no HUD
Indicador CONF. POS no HUD	Frontend	DashboardAMR.jsx	✓ verde ≥ 60%, ■ vermelho < 60%
Badge LIVE / DEMO	Frontend	DashboardAMR.jsx	Verde = WebSocket ativo, Amarelo = fallback mock
Modo VR — WebXR	Frontend	DashboardAMR.jsx	@react-three/xr — botão ENTRAR VR, suporta Quest
Grid tático 3D	Frontend	DashboardAMR.jsx	Linhas de referência + secções 4×4 na cena
WebSocket + reconexão auto	Frontend	DashboardAMR.jsx	Reconecta a cada 5 s se WS cair
Mock fallback (sem backend)	Frontend	DashboardAMR.jsx	Simula patrulha A→B→D→C a cada 4 s
POST /alerta validado	Backend	server.js	Schema validation, enriquece + broadcast WS
LLM report (mock)	Backend	server.js	generateLLMReport() estruturado, pronto p/ API real
Histórico em memória	Backend	server.js	GET /alertas retorna últimas 50 entradas
GET /health	Backend	server.js	Clientes WS, uptime, total alertas
Heartbeat de patrulha	Edge	fire_equipe2.py	Envia evento "normal" + setor atual a cada 3 s
PATROL_PATH 4 setores	Edge	fire_equipe2.py	Rota A→B→D→C em loop (substituível por GPS)
Filtro temporal YOLO	Edge	fire_equipe2.py	CONFIRMATION_FRAMES=5 — anti-falso-positivo
Fallback yolov8n	Edge	download_model.py	Pipeline funciona completo sem best.pt Squad 1
Reestruturação de pastas	Infra	edge/models/	yolov8n movido para edge/models/, styles/ removido

4. Sprint Final — O que falta por Squad

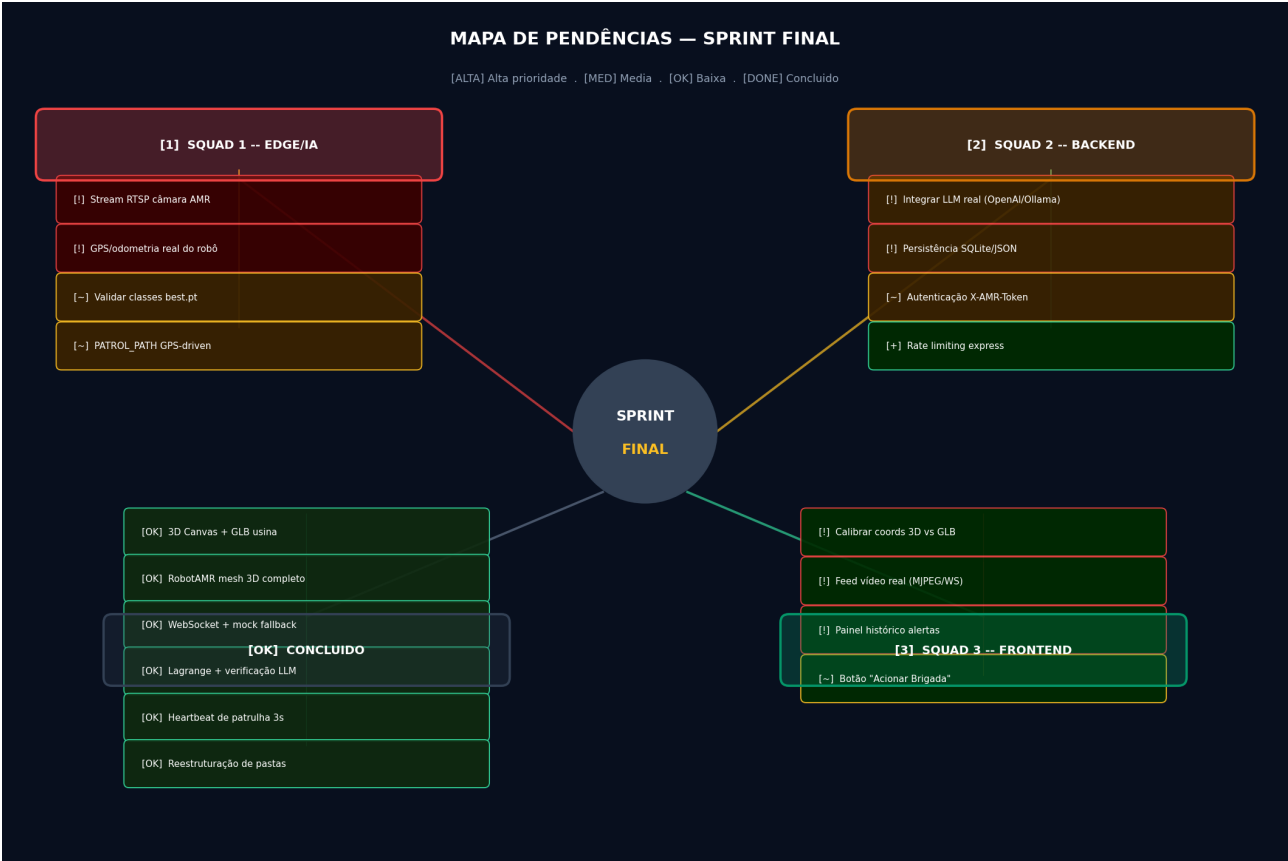


Figura 4 — Mapa de pendências por squad (■ alta · ■ média · ■ baixa)

4.1 Squad 1 — Edge / IA (Python)

Prior.	Tarefa	Descrição Técnica	Est.
■ ALTA	Stream RTSP câmara	Trocar VideoCapture(0) por rtsp://IP_AMR/stream	2 h
■ ALTA	GPS/odometria real	PATROL_PATH via MQTT/Serial da telemetria do AMR	4 h
■ MED	Validar best.pt	Confirmar classes {0:smoke, 1:fire} no modelo Squad 1	1 h
■ MED	PATROL_PATH dinâmico	Receber posição do AMR por API em tempo real	3 h
■ LOW	Log de deteções	Guardar frames com deteções em disco para auditoria	1 h

4.2 Squad 2 — Backend (Node.js)

Prior.	Tarefa	Descrição Técnica	Est.
■ ALTA	LLM real	Substituir generateLLMReport() por OpenAI/Ollama API	3 h
■ ALTA	Persistência	better-sqlite3 ou ficheiro JSON append-only	2 h
■ MED	Autenticação	Header X-AMR-Token fixo no POST /alerta	1 h
■ MED	Rate limiting	express-rate-limit — máx 10 req/min por IP	0.5 h

Prior.	Tarefa	Descrição Técnica	Est.
■ LOW	Endpoint métricas	GET /stats com contagem por evento e intervalo de tempo	1 h

4.3 Squad 3 — Frontend (React + Three.js)

Prior.	Tarefa	Descrição Técnica	Est.
■ ALTA	Calibrar coords 3D	Alinhar SECTOR_TO_3D com geometria real do GLB (Blender)	2 h
■ ALTA	Feed vídeo real	<video> MJPEG ou WebRTC substituindo o CSS placeholder	3 h
■ ALTA	Histórico alertas	Painel lateral consumindo GET /alertas + filtros	2 h
■ MED	Botão "Acionar Brigada"	POST para webhook Slack/Teams/email	1 h
■ MED	Alarme sonoro	new Audio("alarm.mp3").play() em isAlert → true	0.5 h
■ LOW	Snapshot PDF	Botão para exportar estado atual como PDF	2 h

5. Como Rodar o Sistema

Passo	Comando	Resultado Esperado
1. Deps Node	npm install	Instala React, Three.js, Express, WS, Vite
2. Deps Python	pip install -r edge/requirements.txt	Instala ultralytics, opencv-python, requests
3. Modelo IA	python edge/download_model.py	Cria edge/models/best.pt (ou fallback COCO)
4. Tudo junto	npm start	Backend :3000/:3001 + Frontend :5173 (concurrently)
5. Dashboard	http://localhost:5173	Interface 3D/VR — badge DEMO até Python ligar
6. Câmara IA	python -m edge.fire_equipe2	YOLO + heartbeat 3 s → badge muda para LIVE
7. Testar alerta	(usar backend/test_payload.json via curl/Postman)	Robô vai para Setor D e entra em modo ALERT

5.1 Portas e Serviços

Porta	Protocolo	Serviço	Consumido por
3000	HTTP	Express API (POST /alerta · GET /alertas · GET /health)	fire_equipe2.py
3001	WebSocket	Broadcast de telemetria em tempo real	DashboardAMR.jsx
5173	HTTP	Vite Dev Server — React + Three.js	Browser do operador
0	V4L2/DShow	VideoCapture — webcam local ou stream RTSP	fire_equipe2.py